Resumen clases final de Arquitectura de Computadoras 💻

Clase 1: Repaso y Subrutinas.

Arquitectura Von Neumann:

- La unidad central de procesamiento **(CPU)** está constituida por la unidad de control **(UC)** y la unidad aritmético-lógica **(ALU)**.
- Datos e instrucciones deben introducirse en el sistema y los resultados se proporcionarán mediante componentes de entrada/salida **(E/S)**.
- Se necesita almacenar temporalmente datos e instrucciones: **Memoria Principal**.

Elementos de una instrucción:

- Código de operación
- Referencia a operandos fuentes
- Referencia a operando resultado
- Referencia a la siguiente instrucción

Dónde se almacenan los operandos:

1. Memoria principal, memoria virtual, memoria caché
2. Registro de la CPU
3. Dispositivo E/S

Tipos de instrucciones:

- **Procesamiento de datos:**
  - instrucciones aritmético-lógicas
- **Almacenamiento de datos:**
  - instrucciones de memoria
- **Transferencia de datos:**
  - instrucciones E/S
- **Control:**
  - instrucciones de testeo y flujo del programa

Cantidad de direcciones:

- ➔ Más direcciones por instrucción: instrucciones más complejas, más registros. Las operaciones entre los registros son más rápidas. Menos instrucciones por programa.
- ➔ Menos direcciones por instrucción: instrucciones menos complejas, más instrucciones por programa, la captación / ejecución de las instrucciones es más rápida.

Tipos de operandos:

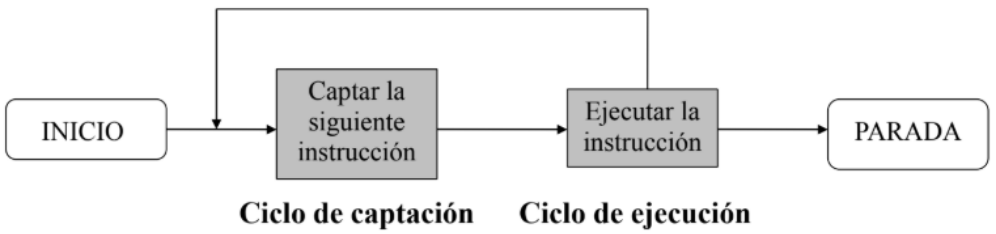
- ➢ Direcciones
- ➢ Números: punto fijo o punto flotante
- ➢ Caracteres
- ➢ Datos lógicos: bits (flags e indicadores)

Modos de direccionamiento:

- Inmediato
- Directo de memoria o absoluto
- Directo de registro
- Indirecto de memoria (no se usa)
- Indirecto con registro
- Indirecto con desplazamiento (usa PC o Pila)

Ciclo de instrucción básico:

1. Captación
2. Ejecución



Ciclo de captación:

La instrucción que se debe captar se encuentra en el Program Counter (PC), luego la unidad de control (UC) capta la instrucción desde la memoria y va al registro de instrucción (IR). Se incrementa el PC y la UC interpreta la instrucción captada y lleva a cabo la acción requerida.

Ciclo de ejecución:

- Transferencia de datos CPU - memoria

- Transferencia de datos CPU y módulo E/S
- Procesamiento de datos: alguna operación aritmética o lógica con los datos
- Control: alteración de la secuencia de ejecución (instrucciones de salto)
- Combinaciones de las anteriores

(Hasta acá es repaso de OC)

Subrutinas: conjunto de instrucciones dentro de un programa que realiza una tarea específica y que puede ser llamada desde otras partes del programa principal o incluso desde otras subrutinas.

- Innovación en lenguajes de programación
- Programa autocontenido
- Puede invocarse desde cualquier punto de un programa (usando CALL)
- Brinda economía (código usado varias veces) y modularidad (subdivisión en unidades pequeñas)
- Requiere pasaje de argumentos (parámetros: por valor, por referencia)

Pasaje de argumentos a subrutinas:

- ➢ Vía registros
  - El número de registros es la principal limitación.
  - Es importante documentar qué registros se usan.
- ➢ Vía memoria
  - Se usa un área de la RAM
  - Es difícil de estandarizar
- ➢ Vía pila (stack)
  - Es el método más usado
  - Es el verdadero “pasaje de parámetros”
  - Independiente de memoria y registros
  - Es usada por el usuario y el sistema

Funcionamiento de Pila:

La pila o stack es una estructura de datos que sigue el principio **LIFO** y se utiliza para gestionar la ejecución de programas y el uso de memoria de manera eficiente. Es un bloque de direcciones de memoria reservado para esta funcionalidad.

- El operando está (de forma implícita) en la cabeza de la pila.
- Se requiere un registro Stack Pointer (SP) que contiene la dirección de la cabeza de la pila.
- Operaciones sobre la pila: **PUSH** apila y **POP** desapila.

- Cuando anidamos múltiples procedimientos o funciones utilizando una pila, se están agregando llamadas a funciones dentro de otras funciones.
- Cada vez que se llama a una nueva función, se agrega un nuevo “marco de pila”, que contiene variables locales, parámetros, la dirección de retorno, y cualquier otra información necesaria para la ejecución de esa función.
- A medida que las funciones anidadas se completan, sus marcos de pila se retiran de la pila, y el control regresa a la función que las llamó.

Procedimientos

Pasos de la llamada a un procedimiento (o subrutina)

1. Salvar el estado de BP *(Base Pointer - puntero base)*
2. Salvar el estado de SP
3. Reservar espacio para datos locales (opcional)
4. Salvar valores de otros registros (opcional)
5. Acceder a parámetros
6. Escribir sentencias a ejecutar
7. Retornar parámetro (opcional)
8. Regresar correctamente del procedimiento

Salida del procedimiento:

- Los registros salvados en la pila deben ser descargados en orden inverso.
- Si se reservó espacio para variables locales, se debe reponer SP con el valor de BP el cual no cambió durante el procedimiento.
- Reponer BP.
- Volver al programa que llamó al procedimiento con **RET**.

Clase 2: Interrupciones.

Interrupción: mecanismo mediante el cual se puede interrumpir el procesamiento normal de la CPU. (Ejecución secuencial de instrucciones de un programa). Estas interrupciones pueden ser de origen interno o externo a la CPU.

Causas de las interrupciones:

- Por un resultado de una instrucción. Por ejemplo: overflow
- Por un temporizador interno del procesador.
- Por una operación E/S. Por ejemplo para indicar la finalización normal de una operación.
- Por un fallo de hardware. Por ejemplo pérdida de energía.

Qué hacer si hubo una interrupción:

En casi todos los casos, implica transferir el control a otro programa o gestor que:

- Salve el estado del procesador
- Corrija la causa que ocasionó la interrupción
- Restaure el estado original del procesador
- Retorne a la ejecución normal del programa interrumpido

➢ No Enmascarables: las que NO pueden ignorarse. Indican eventos peligrosos o de alta prioridad.
➢ Enmascarables: Pueden ser ignoradas. Con instrucciones podemos inhibir la posible solicitud.

Interrupciones por hardware:

- Generadas por dispositivos E/S
- Son las "verdaderas" interrupciones
- El sistema de cómputo tiene que manejar eventos externos "no planeados" o "asincrónicos"
- No están relacionadas con el proceso en ejecución en ese momento.
- Son conocidas como *interrupt request*.

Traps / Excepciones:

Interrupciones por hardware creadas por el procesador en respuesta a ciertos eventos como:

- Condiciones excepcionales: overflow en ALU de punto flotante.
- Falla de programa: tratar de ejecutar una instrucción no definida.
- Fallas de hardware: error de paridad de memoria.
- Accesos no alineados o accesos a zonas de memoria protegidos.

Interrupciones por software:

Las interrupciones por software son interrupciones generadas por una instrucción específica dentro del programa, generalmente para solicitar servicios del sistema operativo. Funcionan de manera similar a las interrupciones por hardware, pero son **provocadas por el programa** y no por algo externo.

❖ El programa no necesita conocer la dirección real de la rutina del sistema.
❖ Las rutinas del sistema operativo pueden ubicarse en cualquier lugar de memoria
❖ Estas interrupciones se utilizan cuando un programa necesita acceder a **servicios del sistema operativo**, por ejemplo funciones del sistema. Esto ocurre porque los programas de usuario no pueden acceder directamente a ciertas partes del sistema operativo, ya que están en memoria protegida.

- ❖ Estas interrupciones se utilizan mediante **una instrucción especial de interrupción.**

Si no tuviéramos las interrupciones por software:

- Los programas deberían conocer las direcciones de memoria de las funciones del sistema operativo o del BIOS para poder llamarlas.
- Como estas direcciones pueden variar, sería necesario modificar el programa cada vez que cambien.
- Las interrupciones por software evitan este problema permitiendo invocar servicios del sistema mediante un número de interrupción en lugar de una dirección específica.

Ciclo de instrucción (con interrupciones):

1. Captación
2. Ejecución
3. Gestión de interrupciones

Mecanismo de interrupción: El mecanismo de interrupción permite a los sistemas manejar eventos en tiempo real, realizar tareas de gestión de dispositivos y responder eficazmente a situaciones imprevistas **sin detener por completo la operación del sistema.**

Estos son sus pasos:

1. **Generar la interrupción:** Un dispositivo o componente del sistema detecta un evento que requiere atención, por lo que genera una señal de interrupción.
2. **Interrupción del procesador:** Cuando se genera una interrupción, el procesador suspende temporalmente la ejecución del programa actual y cambia su enfoque hacia el controlador de interrupciones.
3. **Guarda el estado:** Antes de manejar la interrupción, el procesador guarda el estado actual del programa en ejecución. Esto incluye la posición del programa en la que se detuvo, los registros de la CPU y otros datos relevantes.
4. **Rutina de manejo de interrupciones:** El controlador de interrupciones determina la naturaleza de la interrupción y consulta una tabla de vectores de interrupción que contiene direcciones de memoria de rutinas específicas de manejo de interrupciones. Estas rutinas son porciones de código que se encargan de gestionar el evento específico asociado con la interrupción.
5. **Ejecución de la rutina de manejo:** El procesador comienza a ejecutar la rutina de manejo de interrupciones correspondiente. Esta rutina se encarga de procesar el evento que generó la interrupción. Una vez que la

rutina ha sido ejecutada, el control es devuelto al punto en el programa donde se detuvo inicialmente.

6. **Restauración del estado:** Después de que se maneja la interrupción, el procesador restaura el estado previo guardado, incluyendo los registros y la posición del programa, permitiendo que el programa original se reanude como si la interrupción nunca hubiera ocurrido.
7. **Continuación de la ejecución:** Con el estado restaurado, el programa original continúa su ejecución desde el punto en el que se interrumpió.

Interrupciones múltiples: Situación en la que ocurren varias interrupciones mientras se está manejando una interrupción previa.
Cuando hay múltiples interrupciones puede que al atender una se **inhabiliten las otras**, por lo que las interrupciones se manejan en orden secuencial, o que se hayan definido prioridades (jerarquía de interrupciones).

➔ Interrupciones inhabilitadas
- El procesador puede y debe **ignorar** la señal de petición de interrupción si se produce una interrupción en ese momento.
- Si se hubiera generado una interrupción se **mantiene pendiente** y se examinará luego una vez que se hayan habilitado nuevamente.
- Las interrupciones se manejan en un orden secuencial estricto.

➔ Definir prioridades
- Una interrupción de prioridad **más alta** puede interrumpir a un gestor de interrupción de prioridad menor.
- Cuando se ha gestionado la interrupción de prioridad más alta, el procesador vuelve a las interrupciones previas (de menor prioridad).
- Terminadas todas las rutinas de gestión de interrupciones se **retoma** el programa del usuario.

Características de interrupciones múltiples:

- **Prioridad de interrupciones:** Las interrupciones suelen tener un nivel de prioridad asignado. Cuando se producen múltiples interrupciones, el sistema debe determinar cuál de ellas se manejará primero en función de su prioridad.
- **Máscara de interrupciones:** En algunos sistemas, es posible desactivar temporalmente ciertas interrupciones (de menor prioridad) utilizando máscaras de interrupción.
- **Nivel de anidación:** El sistema debe ser capaz de manejar múltiples niveles de anidación de interrupciones. Si se está manejando una interrupción y ocurre otra de mayor prioridad, el sistema debe ser capaz de suspender temporalmente la primera interrupción, manejar la segunda y luego regresar al manejo de la primera.

- **Protección del estado del sistema:** Durante un manejo de una interrupción, se deben tomar medidas para proteger el estado actual del sistema. Esto generalmente implica guardar los registros y datos críticos antes de comenzar a manejar la interrupción y restaurarlos después de que se haya completado el manejo de la interrupción.

Tratamiento de interrupciones múltiples:

El tratamiento de interrupciones múltiples puede variar, pero en general sigue un enfoque de **manejo de interrupciones en cascada**.

Los pasos que suele seguir son los siguientes:

1. El sistema determina la prioridad de las interrupciones en cola y selecciona la de **mayor prioridad** para manejarla primero.
2. **Se guarda el estado actual del sistema**, incluidos los registros y otros datos relevantes, para que pueda restaurarse más tarde.
3. **Se ejecuta la rutina de manejo de la interrupción** correspondiente a la interrupción seleccionada.
4. Después de que se complete el manejo de la interrupción, **se restaura el estado del sistema que se guardó previamente**.
5. Si aún quedan interrupciones en cola, **el sistema selecciona la siguiente interrupción de mayor prioridad** y repite el proceso.
6. Cuando no quedan interrupciones en cola o cuando se han manejado todas las interrupciones necesarias, **se reanuda la ejecución normal del programa**.

Reconocimiento de interrupciones

- ➔ Interrupciones Multinivel
  - Cada dispositivo que puede provocar una interrupción tiene una entrada física de interrupción conectada a la CPU.
  - Es muy sencillo, pero muy caro.
- ➔ Línea de interrupción única
  - Una sola entrada física de pedido de interrupción a la que están conectados todos los dispositivos.
  - Se debe “preguntar” a cada dispositivo si ha producido el pedido de interrupción (**técnica Polling**).
- ➔ Interrupciones Vectorizadas
  - En las interrupciones vectorizadas, cuando un dispositivo genera una interrupción, también envía un identificador llamado vector de interrupción.
  - Este vector puede ser colocado en el bus de datos por el periférico, o el controlador de interrupciones.

- El procesador usa ese número como índice en la tabla de vectores de interrupción, donde se encuentra la dirección de la rutina de servicio de interrupción (ISR) correspondiente.
- El procesador salta directamente a la rutina que maneja esa interrupción y comienza a ejecutarla.
- Versión resumida: El dispositivo que genera la interrupción envía un vector (identificador) que el procesador usa para buscar en la tabla de vectores la dirección de la rutina de servicio de interrupción, saltando directamente a ella.

Controlador de Interrupciones

La función de un controlador de interrupciones es gestionar y coordinar las interrupciones. Se encarga de determinar qué dispositivo o evento generó la interrupción, priorizarlas en caso de múltiples interrupciones simultáneas y luego redirigir el flujo de ejecución del procesador hacia un **programa de manejo de interrupciones** específico.

PIC: Dispositivo controlador programable de interrupciones

Se utiliza para gestionar múltiples fuentes de interrupción en un sistema.
Su función principal es recibir interrupciones de varios dispositivos y enviarlas al procesador a través de una o pocas líneas de interrupción.
Además, el PIC puede asignar prioridades entre las distintas interrupciones para decidir cuál debe atender primero el procesador.

Funciones principales:

- Recibir interrupciones de varios dispositivos
- Priorizar las interrupciones cuando ocurren varias al mismo tiempo.
- Enviar la interrupción a la CPU usando una línea de interrupción.

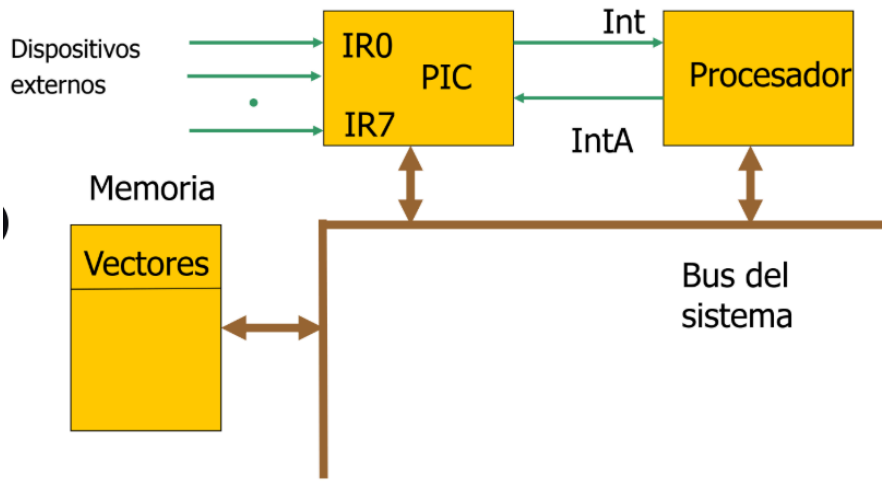


Tabla de vectores de interrupción: La tabla de vectores de interrupción es una **estructura en memoria** que relaciona cada tipo de interrupción con la **dirección** de la rutina que debe atenderla.
Cada entrada de la tabla contiene la dirección de memoria de la rutina de servicio de interrupción correspondiente.
En muchas arquitecturas, cada entrada ocupa 4 bytes (una doble palabra).

Vectores preasignados:

- Tipo 0: **finaliza la ejecución** del programa
- Tipo 3: **punto de parada** para depuración / seguimiento
- Tipo 6: **lectura** de entrada estándar, requiere uso de BX
- Tipo 7: **escritura** de salida estándar, requiere BX y AL.

Registros internos **PIC**: Se sitúan a partir de la dirección 20H, son accedidos con operaciones de lectura y escritura en el espacio de E/S.

- **EOI:** (End of Interruption): Fin de interrupción. Como consecuencia, se pone en 0 el bit del ISR correspondiente. Sirve para indicarle al PIC que ya fue atendida la interrupción y la CPU vuelve a la ejecución normal del programa.
- **IMR:** (Interrupt Mask Register): Registro de máscara de interrupciones, permite el enmascaramiento selectivo de cada una de las entradas de interrupción, indicando con bit en 1 (Indica cuáles deben ser ignoradas). Tras un reset los bits de este registro quedarán en 0.
- **IRR:** (Interrupt Request Register): Registro de petición de interrupción, indica con bit en 1 las interrupciones demandadas hasta el momento.
- **ISR:** (In Service Register): Registro de interrupción en servicio, indica con bit en 1 cuál es la interrupción que está siendo atendida.
- **INT0...INT7:** 8 registros, donde se cargan los id de interrupciones. Cada uno con su vector.

Funcionamiento del PIC

1. **Detección de interrupciones:** El PIC monitorea constantemente las líneas de interrupción de los dispositivos periféricos y otros eventos en busca de cambios en su estado. Cuando se detecta una interrupción, se genera una solicitud de interrupción hacia el procesador.
2. **Priorización:** Si hay múltiples interrupciones pendientes, el PIC selecciona la de mayor prioridad según su configuración. Esto se basa en las prioridades asignadas a las líneas de interrupción.
3. **Enmascaramiento:** Antes de notificar al procesador sobre una interrupción, verifica si la línea de interrupción está habilitada o enmascarada. Si la línea está enmascarada, la interrupción se ignora.

4. **Vectorización:** Una vez que se determina la interrupción que debe atenderse, el PIC envía un vector de interrupción a la CPU. Este vector contiene información sobre la fuente de la interrupción y apunta a la dirección de memoria de la rutina de manejo de interrupciones correspondiente.
5. **Procesamiento de interrupción:** La CPU recibe el vector de interrupción y ejecuta la rutina de manejo de interrupciones asociada. Esta rutina es la encargada de gestionar la interrupción, guardar el estado necesario y realizar cualquier acción requerida.
6. **Fin de interrupción (EOI):** Después de manejar la interrupción, la CPU envía una señal de Fin de Interrupción (EOI) al PIC para indicar que ha completado el manejo de la interrupción actual. Esto permite que el PIC se prepare para la siguiente interrupción si la hubiera.

¿Cuáles son las diferencias entre la terminación de una subrutina y un gestor de interrupción?

- La subrutina regresa al programa principal utilizando la instrucción de retorno **RET.**
- En cambio, un gestor de interrupción lo hace después de manejar dicha interrupción, o con la instrucción de retorno **IRET.**
- La instrucción IRET es similar a una instrucción RET, por utilizar la pila, pero recupera una copia del registro de estado y la dirección de retorno

---

Clase 3: Entrada / Salida.

Problemas de Entrada/Salida

- Gran variedad de periféricos con varios métodos de operación: cantidad de datos, velocidad de transmisión y distintos formatos de datos.
- Todos más lentos que la CPU y la RAM
- **Necesidad de módulos de E/S**

Módulo de E/S:

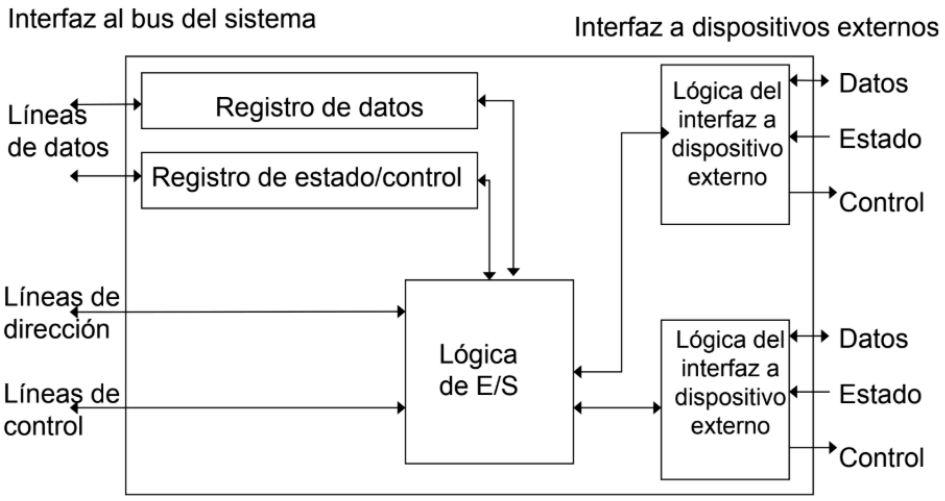
Realiza la interfaz entre el procesador, la memoria y los periféricos, pueden manejar uno o más periféricos.
Un módulo de E/S es un componente clave que permite la comunicación entre la CPU y dispositivos periféricos.
El funcionamiento de un módulo de E/S permite que el procesador vea una amplia gama de dispositivos de una **forma simplificada** que oculta los detalles de temporización, formatos y electromecánica propios de los dispositivos para que el

procesador pueda funcionar únicamente con órdenes de lectura / escritura o abrir / cerrar ficheros.

Funciones de un módulo de E/S

- Control y temporización de uno o más dispositivos externos
- Ocultar las propiedades del dispositivo a la CPU
- Interpretar las órdenes que recibe de CPU y transmitirlas al periférico
- Comunicación con la CPU (registros) y memoria
- Controlar las transferencias de datos entre CPU y el periférico (convertir formatos, adaptar velocidades)
- Comunicación con los dispositivos (periféricos)
- Informar a la CPU del estado del periférico
- Almacenamiento temporal (buffering) de datos
- Detección de errores.

Diagrama en bloques de un módulo E/S:



Estructura de un módulo de E/S:

- ➔ El módulo se conecta al resto de la computadora a través de un conjunto de líneas (por ejemplo, líneas del bus del sistema).
- ➔ Los datos que se transfieren a y desde el módulo se almacenan temporalmente en uno o más registros de datos.
- ➔ Puede haber uno o más registros de estado que proporcionan información del estado presente. **Un registro de estado también puede funcionar como un registro de control**, para recibir la información de control del procesador.
- ➔ La lógica que hay en el módulo interactúa con el procesador a través de una serie de líneas de control, éstas son las que utiliza el procesador para proporcionar órdenes al módulo de E/S.

- ➔ El módulo también debe ser capaz de reconocer y generar las direcciones asociadas a los dispositivos que controla.
- ➔ Cada módulo de E/S tiene una dirección única o, si controla más de un dispositivo externo, un conjunto único de direcciones.
- ➔ El módulo de E/S posee la lógica específica para la interfaz con cada uno de los dispositivos que controla.

Direccionamiento de E/S: modos de ubicación de los módulos E/S

Los módulos de E/S pueden ser ubicados en direcciones asociadas a la **E/S mapeada en memoria y la E/S aislada**

- ❖ E/S mapeada en memoria *(memory-mapped)*
  - ➢ Los dispositivos de E/S y memoria comparten un único espacio de direcciones.
  - ➢ Tendremos una cierta cantidad de direcciones de memoria asignada a las operaciones de E/S
    - No hay necesidad de manejar instrucciones distintas
  - ➢ No hay órdenes específicas para E/S:
    - Hay variedad de órdenes de acceso a memoria
    - Programación más eficiente
  - ➢ La E/S se parece a la memoria de lectura/escritura

- ❖ E/S aislada
  - ➢ Tenemos espacios de direcciones separados
    - Tendremos direcciones de memoria principal y por otro lado tendremos direcciones asociadas a las E/S
  - ➢ Necesidad de líneas especiales de E/S y de memoria.
  - ➢ Necesidad de órdenes específicas para E/S:
    - Conjunto limitado de instrucciones

Técnicas de gestión de E/S:

- **E/S programada con espera de respuesta**
- **E/S con interrupciones**
- **E/S con acceso directo a memoria** (DMA)

E/S Programada ⏳

- Intercambio de datos entre la CPU y el módulo
- La CPU tiene control directo sobre la operación de E/S
  - Comprobación del estado del dispositivo
  - Envío de comandos de lectura/escritura
  - Transferencia de datos

- La CPU espera que el módulo E/S termine la operación, por lo tanto permanece activa (trabajando) durante un período de tiempo (esto es malo).
- El procesador ejecuta un programa que controla directamente la operación de E/S, y debe esperar hasta que la operación concluya. Si el procesador es más rápido que el módulo de E/S, el procesador desperdicia este tiempo.

Etapas de E/S Programada ⏳

1. La CPU solicita la operación de E/S al módulo.
2. El módulo E/S realiza la operación
3. El módulo E/S activa los bits de estado del dispositivo direccionado y espera.
4. La CPU comprueba periódicamente el estado de esos bits, hasta que detecta que la operación fue completada.
5. Si no fue completada, espera y vuelve a comprobarlo más tarde.

E/S con Interrupciones 🛑

- La CPU no tiene que esperar la finalización de la tarea de E/S, puede seguir procesando.
- No se repite la comprobación de los estados de los módulos.
- El módulo envía un pedido de interrupción a la CPU cuando está listo nuevamente.

Etapas de E/S mediante interrupciones: 🛑

1. La CPU envía una orden de lectura (READ)
    - El módulo E/S obtiene los datos del periférico mientras que la CPU realiza otro trabajo
2. Luego chequea si hay pedidos de interrupciones pendientes al final de cada ciclo de instrucción.
    - El módulo E/S emite un pedido de interrupción a la CPU
3. La CPU detecta el pedido, guarda el contexto, interrumpe el proceso y realiza la gestión de la interrupción.
4. Solicita los datos
    - Módulo E/S transfiere los datos.

Interrupciones múltiples 🛑

- Todas las líneas de interrupción tienen un orden de prioridad
- Las líneas con más prioridad pueden interrumpir a las líneas con menor prioridad.
- Si existe un maestro del bus, solo él puede interrumpir.

- Las interrupciones de E/S mediante interrupciones son más efectivas que las programadas, pero ambas necesitan intervención directa de la CPU
    - La velocidad de transferencia es limitada
    - La CPU permanece ocupada mucho tiempo durante la operación

Acceso Directo a Memoria (DMA) 💿

El DMA *(Direct Memory Access)* es un mecanismo de entrada/salida que permite **transferir datos directamente entre un dispositivo de E/S y la memoria principal sin intervención continua de la CPU.**

Para ello se utiliza un controlador de DMA (DMAC), que se encarga de gestionar la transferencia.

Funcionamiento del DMA:

Cuando el procesador necesita transferir un bloque de datos entre memoria y un dispositivo:

1. La CPU configura el controlador de DMA, indicando:
    - el tipo de operación (lectura o escritura)
    - la dirección del dispositivo de E/S,
    - la dirección inicial de memoria donde se leerán o escribirán los datos,
    - la cantidad de datos a transferir.
2. Una vez configurado, la CPU continúa ejecutando otros programas.
3. El controlador de DMA solicita el control del bus para realizar la transferencia.
4. Cuando obtiene el bus, el DMAC transfiere los datos directamente entre el dispositivo y la memoria, palabra por palabra, sin que la CPU intervenga.
5. Cuando la transferencia termina, el DMAC envía una interrupción a la CPU para indicarle que la operación finalizó.

Usos del DMA: El DMA se utiliza cuando se deben transferir **grandes cantidades de datos** entre dispositivos y memoria, ya que permite:

- reducir la intervención de la CPU
- mejorar el rendimiento del sistema
- liberar al procesador para ejecutar otros procesos

Tipos de transferencias

- ➢ Si el **DMAC** solo toma el control del bus durante los intervalos de tiempo en los que la CPU no lo usa, **el rendimiento del sistema no sufrirá degradación alguna.**
- ➢ Dos tipos de transferencia:
    - Por **ráfagas** (burst)
    - Por **robo de ciclo** (cycle-stealing)

DMA modo ráfaga (burst):

El DMAC solicita el control del bus a la CPU, cuando ésta lo concede, el DMAC no lo libera hasta haber finalizado la transferencia de **todo el bloque de datos completo.**

- ★ **Ventajas:** la transferencia se realiza de forma rápida.
- ★ **Desventajas:** durante el tiempo que dura la transferencia la CPU no puede utilizar el bus, lo que puede degradar el rendimiento del sistema.

DMA modo robo de ciclo (cycle-stealing):

El DMAC solicita el control del bus a la CPU, cuando ésta lo concede, se realiza la transferencia de **una única palabra** y después el DMAC libera el bus.
El DMAC solicita el control del bus **tantas veces como sea necesario** hasta finalizar la transferencia del bloque completo.

- ★ **Ventajas:** no se degrada el rendimiento del sistema.
- ★ **Desventajas:** la transferencia tarda más tiempo en llevarse a cabo.

No cuenta como interrupción ya que no se guarda el contexto. El trabajo de la CPU es lento pero no tanto como si ella realizara la transferencia, por lo tanto para transferencias de E/S de múltiples palabras, es la técnica más eficiente.

Etapas de una transferencia DMA 💿

El Acceso Directo a Memoria permite que un dispositivo periférico acceda a memoria principal (RAM) directamente sin la intervención de la CPU.
El DMA acelera la transferencia de datos entre la memoria y los dispositivos periféricos, lo que libera recursos de la CPU para otras tareas. Las características funcionales del DMA incluyen varias etapas de transferencia:

1. Inicialización de la transferencia
    - La primera etapa implica que un dispositivo periférico envíe una solicitud de acceso a la memoria al controlador DMA
    - La CPU debe enviar a la interfaz del periférico y al DMAC los parámetros de la transferencia.
2. Selección del Canal DMA:
    - En esta etapa, el sistema debe asignar el canal DMA a la solicitud entrante.
3. Configuración:
    - Una vez seleccionado el canal DMA, se configura para que coincida con los requisitos de la transferencia de datos.
    - Configuración de las direcciones de inicio y finalización en la memoria, el tamaño de la transferencia y otras características relevantes.
4. Acceso Directo:

    - El controlador DMA se comunica directamente con la memoria principal y el dispositivo periférico para iniciar la transferencia de datos.
    - La CPU no participa en la transferencia en sí, lo que permite que la CPU realice otras tareas mientras se lleva a cabo la transferencia
5. Realización de la transferencia
    - El controlador DMA transfiere datos entre la memoria y el dispositivo periférico utilizando el canal DMA configurado previamente.
    - Cuando el periférico está listo para realizar la transferencia se lo indica al DMAC
    - El DMAC pide el control del bus y se realiza la transferencia entre el periférico y la memoria.
    - La transferencia puede ser en una dirección o bidireccional.
6. Finalización de la transferencia
    - Cuando se completa la transferencia, el controlador DMA notifica al dispositivo periférico y actualiza cualquier estado relevante.
    - El DMAC libera el bus y devuelve el control a la CPU
    - El DMAC suele activar una señal de interrupción para indicar a la CPU la finalización de la operación de E/S solicitada.
7. Liberación del canal DMA:
    - Después de completar la transferencia, el canal DMA se libera para su uso posterior
    - Esto permite que otros dispositivos o solicitudes utilicen el canal DMA según sea necesario.

- Después de la inicialización la CPU retorna sus tareas y ya no se preocupa más de la evolución de la transferencia.

Problemas que pueden haber 💿

- El uso intensivo del DMA puede degradar el rendimiento de la CPU, ya que cuando el DMAC utiliza el bus la CPU no puede acceder a la memoria. Este problema se reduce con el **uso de memoria caché**, porque la CPU puede obtener muchas instrucciones desde la caché sin usar el bus, permitiendo que el DMAC aproveche esos intervalos para realizar transferencias.
- **Coherencia de datos:** La coherencia de datos puede verse afectada por el uso de DMA, ya que si se realizan operaciones sobre un dato, éste se actualizará en caché y si no se vacía en memoria antes de que un dispositivo intente accederlo, se podría estar utilizando un valor erróneo, y viceversa.

---

esto no vi que lo tomen

Canales de E/S

1. La CPU controla directamente los periféricos
2. Se agrega un módulo de E/S o controlador
3. El módulo de E/S provee el acceso directo a memoria (DMA)
4. El módulo de E/S tiene su propio procesador con su pequeño conjunto de instrucciones y memoria local

Características:

- Representan una extensión al concepto de DMA
- Tienen la habilidad de ejecutar instrucciones de E/S
- Completo control de la transferencia de datos.
- Programa almacenado en memoria principal.

Tipos:

- ➢ Selector:
  - Controla varios dispositivos de alta velocidad uno a la vez, por lo tanto el canal se dedica a la transferencia de datos de ese dispositivo.
  - El canal selecciona un dispositivo y efectúa la transferencia.
  - Los dispositivos son manejados por un controlador o módulo de E/S.
- ➢ Multiplexor:
  - Puede manejar E/S con varios dispositivos a la vez.
  - Multiplexor de bytes: acepta y transmite caracteres.
  - Multiplexor de bloques: intercala bloques de datos desde distintos dispositivos.

Clase 4: Segmentación de Instrucciones.

Segmentación de cauce:

- La segmentación de cauce *(pipelining)* es una forma particularmente efectiva de **organizar el hardware de la CPU para realizar más de una operación al mismo tiempo**.
- **Consiste en descomponer el proceso de ejecución de las instrucciones en fases o etapas que permitan una ejecución simultánea.**
- Explota el **paralelismo** entre las instrucciones de flujo secuencial.

Características de la segmentación:

- La técnica es a nivel **hardware**
- Es invisible al programador
- Depende del repertorio de instrucciones

- Uniformizar etapas al tiempo de la más lenta

Prestaciones del cauce segmentado:

- **Mejora el rendimiento:** La ejecución en paralelo acelera la velocidad de ejecución de las instrucciones.
- **Aprovechamiento de recursos:** Mientras una unidad de ejecución realiza una operación, las etapas anteriores y posteriores pueden estar ocupadas con otras instrucciones, lo que permite un uso más completo de las unidades funcionales.
- **Mayor paralelismo:** Al permitir que múltiples instrucciones se ejecuten al mismo tiempo, la segmentación de cauce aumenta el nivel de paralelismo a nivel de instrucciones en un procesador.
- **Reducción del ciclo de reloj por instrucción:** Reduce el tiempo necesario para ejecutar una instrucción completa al dividirla en etapas más pequeñas.
- **Mejora de la predicción de saltos:** Puede combinarse con técnicas de predicción de saltos para reducir las penalizaciones por fallos de predicción de saltos, lo que resulta en una ejecución más eficiente en las instrucciones.

Limitaciones al paralelismo a nivel de instrucciones

- **Dependencias de datos:** Ocurren cuando una instrucción necesita el resultado de otra anterior. Esto obliga a esperar, reduciendo el paralelismo.
- **Acceso a memoria:** El acceso a memoria es más lento que las operaciones internas del CPU. Si hay muchos accesos, el procesador debe esperar, limitando la ejecución en paralelo.
- **Limitaciones del hardware:** El paralelismo está limitado por la arquitectura del procesador, como:
  - cantidad de unidades funcionales
  - tamaño de la ventana de ejecución

El paralelismo a nivel de instrucciones está limitado principalmente por **dependencias entre instrucciones, saltos en el flujo de control, accesos a memoria y restricciones del hardware (tamaño de la ventana de ejecución).**

Clase 5: Posibles soluciones a atascos

Atascos de un cauce (stall): situaciones que impiden a la siguiente instrucción que se ejecute en el ciclo que le corresponde.

- **Estructurales:**

    - Son provocadas por conflictos por los recursos, dos o más instrucciones necesitan utilizar el **mismo recurso de hardware** en el mismo ciclo.
    - Acceso simultáneo a memoria si es unificada

- **Por dependencia de datos:**
    - Ocurre cuando dos instrucciones se comunican por medio de un **dato.**
    - Condición en la que los operando fuente o destino de una instrucción no están disponibles en el momento en que se necesitan en una etapa determinada del cauce.
    - Tipos de dependencias de datos:
        - **Lectura después de escritura (RAW, dependencia verdadera):** una instrucción genera un dato que lee otra posterior
        - **Escritura después escritura (WAW, dependencia en salida):** una instrucción escribe un dato después que otra posterior. Sólo se da si se deja que las instrucciones se adelanten unas a otras
        - **Escritura después lectura (WAR, antidependencia):** una instrucción modifica un valor antes de que otra anterior que lo tiene que leer, lo lea. No se puede dar en nuestro cauce simple.

- **Por dependencia de control (o de instrucciones):** saltos
    - Situación en la que la ejecución de instrucciones en un procesador segmentado se ve afectada negativamente debido a saltos de programa, como instrucciones de salto condicional o incondicional.
    - Estos riesgos pueden dar lugar a retrasos en la ejecución de instrucciones y pueden afectar el rendimiento general del procesador.

Soluciones a riesgos estructurales

Simple: Replicar, segmentar o realizar turnos para el acceso a las unidades funcionales en conflicto.

- Duplicación de recursos de hardware
    - Sumadores o restadores además de la ALU
- Separación en memorias de instrucciones y datos
- Turnar el acceso al banco de registros
    - Escrituras en la primera mitad de los ciclos de reloj
    - Lecturas en la segunda mitad de los ciclos de reloj

Soluciones a riesgos de datos

Para riesgos RAW: se debe determinar cómo y cuándo aparecen esos riesgos. **Será necesaria una unidad de detección de riesgos y/o compilador más complejo.**

Dos soluciones:

- Técnica Hardware: Adelantamiento de operandos (forwarding)
  - Consiste en pasar directamente el resultado obtenido con una instrucción a las instrucciones que lo necesitan como operando.
    - Si el dato necesario está disponible a la salida de la ALU se lleva a la entrada de la etapa correspondiente sin esperar a la escritura.
  - Fácil de implementar si se identifican todos los adelantamientos y se comunican a los registros de segmentación correspondientes.

- Técnica Software: Instrucciones NOP o reordenación de código
  - Evita los riesgos reordenando las instrucciones del código sin afectar los resultados
  - Realizada por el compilador
    - Introducción de instrucciones NOP
      - Se genera retardo
    - Reordenación de instrucciones
      - Máxima separación de instrucciones con dependencia RAW
      - Cuidado con ejecución “fuera de orden”

Soluciones a riesgos de control: Existe una penalización por salto 🔶

La ejecución de instrucciones en un procesador segmentado se ve afectada negativamente debido a saltos de programa, como **instrucciones de salto condicional o incondicional.**
Estos riesgos pueden dar lugar a retrasos en la ejecución de instrucciones y pueden afectar el rendimiento general del procesador.

- Instrucciones de salto

  - Incondicional
    - La dirección de destino se debe determinar lo más pronto posible, dentro del cauce, para reducir penalización.
  - Incondicional tomado:
    - Cuando la condición se resuelve como verdadera, lo que significa que se debe tomar el salto, las instrucciones que siguen al salto condicional y ya han avanzado deben descartarse, lo que conduce a un desperdicio de ciclos de reloj y una penalización en el rendimiento.

- ➢ Incondicional no tomado:
  - Cuando la condición se resuelve como falsa, lo que significa que no se toma el salto, las instrucciones que siguen al salto deben continuar su ejecución.
- ➢ Condicional
  - Introduce riesgo adicional por la dependencia entre la condición de salto y el resultado de una instrucción previa.
  - Ocurre cuando se encuentra una instrucción de salto incondicional. Las instrucciones que siguen ya han avanzado en el cauce y se encuentra en diferentes etapas de ejecución.
  - Cuando se toma el salto, estas instrucciones en etapas posteriores deben ser descartadas, lo que resulta en un desperdicio de ciclos de reloj y una penalización en el rendimiento.

❖ Tratamiento de saltos

- ➢ Técnica Hardware
  - Predicción de saltos para evitar la parada
- ➢ Técnica Software
  - Salto retardado o de relleno de ranura de retardo
    - El compilador introduce instrucciones que se ejecutarán en cualquier caso después de la instrucción de salto

Predicción de saltos (técnica hardware)

- Técnicas estáticas
  - Predecir que nunca salta:
    - Asume que el salto no se producirá
    - Siempre capta la siguiente instrucción
  - Predecir que siempre se salta:
    - Asume que el salto se producirá
    - Siempre capta la instrucción destino del salto

- Técnicas dinámicas
  - Conmutador saltar / no saltar:
    - Basado en la historias de las instrucciones
    - Eficaz para los bucles
  - Tabla de historia de saltos *(branch-target buffer)*
    - Pequeña caché asociada a la etapa de búsqueda *(fetch)*
    - Tres campos:
      1. Dirección de una instrucción de bifurcación
      2. Información de la instrucción destino (dirección del destino o instrucción destino)

3. N bits de estado (historia de uso)

- Otras soluciones Hardware
  - Predecir según el código de operación
    - Hay instrucciones con más probabilidades de saltar
    - La tasa de acierto puede llegar a alcanzar un 75%
  - Flujos múltiples
  - Precaptar el destino del salto
  - Buffer de bucles

Flujos múltiples

- Varios cauces (uno por cada opción de salto)
- Precaptan cada salto en diferentes cauces
- Se debe utilizar el cauce correcto
- Desventajas:
  - Provoca retardos en el acceso al bus y a los registros.
  - Si hay múltiples saltos, se necesita un mayor número de cauces.

Precaptación del destino de salto

- Se precapta la instrucción destino del salto, además de las instrucciones siguientes a la bifurcación.
- La instrucción se guarda hasta que se ejecute la instrucción de bifurcación

Buffer de bucles

- Memoria muy rápida. Gestionada por la etapa de captación de instrucción del cauce.
- Comprueba el buffer antes de hacer la captación de memoria.
- Muy eficaz para pequeños bucles y saltos

Salto retardado (Técnica Software)
Idea: realizar trabajo útil mientras el salto se resuelve

- **Hueco o ranura de retardo de salto** *(delay-slot)* es el período de penalización o parada luego de una instrucción de salto.
- El compilador trata de situar instrucciones útiles (que no dependan del salto) en los huecos de retardo. Si no es posible, se utilizan instrucciones NOP.
- **Las instrucciones en los huecos de retardo de salto se captan siempre.**
- Requiere reordenar las instrucciones.

Clase 6: RISC y CISC

¿Qué es CISC? *(Complex Instruction Set Computer):*

Arquitectura basada en un conjunto grande de **instrucciones complejas**, donde cada instrucción puede realizar **varias operaciones**.

CISC busca hacer **más trabajo por instrucción,** pero a costa de **mayor complejidad y menor eficiencia** en la ejecución.

Objetivo:

- ➔ Facilitar el trabajo del compilador
- ➔ Reducir la cantidad de instrucciones por programa
- ➔ Acercarse más a los lenguajes de alto nivel (HLL)

Características

- Muchas instrucciones
- Instrucciones complejas
- **Varios modos de direccionamiento**
- Algunas instrucciones realizan varias tareas en una sola

Desventajas

- Mayor complejidad del hardware
- Instrucciones más lentas (pueden requerir varios ciclos)
- **Difícil de optimizar**
- No siempre mejora realmente la velocidad

De estas desventajas surge la idea de RISC, que propone:

- ★ **Instrucciones simples**
- ★ **Ejecución más rápida**
- ★ **Mejor optimización**

RISC: *Reduced Instruction Set Computer*

RISC busca simplificar las instrucciones para lograr una **ejecución más rápida, eficiente y predecible.**

Características

- ➢ **Conjunto de instrucciones reducido y simple**

- Instrucciones simples, rápidas y de tamaño fijo
- Generalmente una instrucción por ciclo
- Operaciones registro a registro
- **Modos de direccionamiento simples**
- Gran cantidad de registros de uso general
- Diseño sin microcódigo (cableado)

RISC vs. CISC

- No existe una clara barrera diferenciadora
- **Muchos diseños incluyen características de ambos criterios**

No es fácil comparar RISC vs CISC porque:

- No hay máquinas directamente equivalentes
- Depende mucho del compilador
- Muchas arquitecturas actuales son híbridas

Las arquitecturas **RISC** se caracterizan por un conjunto de instrucciones simple, instrucciones de longitud fija y una ejecución más simple, lo que las hace adecuadas para una **ejecución más eficiente y predecible.**
Las arquitecturas **CISC**, por otro lado, tienen un conjunto de instrucciones más complejo y versátil, lo que las hace adecuadas para **tareas más variadas** pero a costa de una mayor complejidad de hardware y menos previsibilidad en el tiempo de ejecución.

---

más largo y complejo:

Controversia RISC y CISC

- **Cuantitativa:**
  - Comparación del tamaño de los programas y su velocidad de ejecución
- **Cualitativa:**
  - Revisión de soporte de lenguajes de alto nivel y uso óptimo de los recursos VLSI *(Very Large Scale Integration - Integración a Muy Gran Escala):* se refiere a qué arquitectura usa mejor el espacio físico y la complejidad permitida por la tecnología del chip.
- **Problemas de las comparaciones:**
  - No existe un par de máquinas RISC y CISC directamente comparables
  - No hay un conjunto de programas de prueba definitivo
  - Dificultad para separar los efectos del hardware del compilador
  - Mayoría de comparaciones con máquinas de “juguete”, no con productos comerciales
  - La mayoría de las máquinas son una mezcla de ambas

- ❖ **CISC** usaba muchas instrucciones complejas → requería más hardware y más microcódigo.
- ❖ **RISC** apostaba por pocas instrucciones simples → menos circuitería, más espacio para optimizaciones como *pipelines* o *caches*.

Clase 7: Memoria

Sistema de Memoria

Jerarquía de memoria: acceder a cantidades ilimitadas de memoria rápida

- Organizada en niveles que son ubicados en distintos lugares físicos.
- Fabricados con tecnologías diferentes que se gestionan de manera independiente.

- ❖ **Objetivo:** la velocidad del sistema deberá ser, aproximadamente, la del nivel más rápido al costo del nivel más barato
- ❖ A medida que nos alejamos de la CPU, cada nivel inferior es más grande, más lento y más barato que el nivel previo (o superior) en la jerarquía.
- ❖ Debe haber correspondencia de direcciones en los distintos niveles.
- ❖ Propiedades a cumplir:
  - ➢ **Inclusión:** los datos almacenados en un nivel han de estar almacenados en los niveles inferiores a él
  - ➢ **Coherencia:** las copias de la misma información en los distintos niveles deben contener los mismos valores.

Principio de localidad de referencia: hace que funcione la jerarquía

- ➔ **Localidad Temporal:** los elementos de memoria referenciados recientemente (datos o instrucciones), volverán a serlo en un futuro próximo.
  - ◆ Subo la **palabra** de nivel
- ➔ **Localidad Espacial:** los elementos de memoria cuyas direcciones están próximas a los últimos referenciados serán referenciados.
  - ◆ Subo un **bloque** (con la palabra) de nivel

Memoria Caché

- Cantidad pequeña de memoria rápida
- Se ubica entre la memoria principal y la CPU
- Puede localizarse en un chip o en módulo CPU

Dos niveles de caché: El uso de dos niveles de caché (L1 y L2) es una característica clave en la jerarquía de memoria. La jerarquía de memoria tiene el objetivo de brindar la mayor cantidad de espacio, la velocidad más rápida y al menor costo posible. Esto lo logra combinando distintos niveles, cada uno independiente y usando tecnología diferente.

Cuando se separa la memoria caché en dos, se tiene por un lado la **caché L1** (nivel uno), que se encuentra en un sector interno **fácil de acceder para el procesador**, y la **L2**, más grande que L1 pero con **acceso un poco más lento**. Si no se encuentra un dato en L1, se chequea si existe en L2, es una forma de evitar aún más los costosos accesos a memoria principal

- Reducción del tiempo de acceso promedio
- Aprovechamiento de la localidad de referencia
- Optimización de costos y consumo de energía
- Reducción de la latencia de acceso a la memoria principal
- Mayor escalabilidad

Número de cachés: Hay dos aspectos relacionados con este tema, el número de niveles de caché, y el uso de caché unificada frente al de cachés separadas.

- ★ **Cachés multinivel:** En general, el uso de un segundo nivel de caché mejora las prestaciones. No obstante, el uso de cachés multinivel complica todos los aspectos de diseño de la caché.
- ★ **Caché unificada frente a cachés separadas:** Muchos diseños contenían una sola caché para almacenar las referencias tanto a datos como a instrucciones. Más recientemente, se ha hecho normal separar la caché en dos: **una dedicada a instrucciones y otra a datos.** La ventaja clave del diseño de una caché partida es que elimina la competición por la caché entre el procesador de instrucciones y la unidad de ejecución. Esto es importante en diseños que cuentan con **segmentación de cauce de instrucciones.**

Funcionamiento de la cache

- La CPU solicita contenido de una dirección de memoria
- Si la caché tiene ese dato lo obtiene inmediatamente
- Si no está el dato, se lee el bloque que contiene esa dirección desde la memoria principal y copia en la caché. Se busca el bloque en los siguientes niveles de la jerarquía de memoria hasta resolver el fallo
    - Después la caché entrega el dato requerido a la CPU

La cache incluye etiquetas para identificar qué bloque de la memoria principal está en cada una de sus líneas.

Tiempo de acceso medio a memoria:

$$T_{acceso} = T_{acierto} + TF \; x \; PF$$

Para mejorar las prestaciones:

- Reducir el tiempo en caso de acierto ($T_{acierto}$)
- Reducir tasa de fallos (TF)
- Reducir la penalización por fallo (PF)
- **Aumentar el tamaño de la memoria caché** permite que más datos se almacenen en ella, lo que reduce la necesidad de acceder a la memoria principal con frecuencia.
- Utilizar una **política de reemplazo adecuada** puede ayudar a mantener los datos más relevantes en la caché y reducir los fallos de caché.
- Optimizar los algoritmos y estructuras de datos utilizados en el software para **aprovechar la localidad espacial y la localidad temporal**.

Ubicación de un bloque / Tipos de correspondencia

Se necesita un algoritmo que haga corresponder bloques de memoria principal a líneas de caché. La elección de la función de correspondencia **determina cómo se organiza la caché**. Pueden utilizarse tres técnicas:

- Correspondencia **directa**: Un bloque solo puede estar almacenado en un lugar de la caché.
- Correspondencia **totalmente asociativa**: Un bloque puede almacenarse en cualquier lugar de la caché.
- Correspondencia **asociativa por conjuntos**: Un bloque puede almacenarse en un conjunto restringido de lugares en la caché.

Correspondencia Directa: ventajas y desventajas

- Simple
- Poco costosa
- Hay una posición concreta para cada bloque dado:
    - Si un programa accede a dos bloques que se corresponden a la misma línea (diferentes bloques de memoria principal) de forma repetida, las pérdidas de caché (desaciertos) serán muy grandes.

Correspondencia Asociativa: ventajas y desventajas

- Un bloque de memoria principal puede colocarse en cualquier línea de la caché
- La etiqueta identifica unívocamente un bloque de memoria
- Todas las etiquetas de las líneas se examinan para buscar una coincidencia
- Búsqueda costosa (en tiempo principalmente)

Correspondencia asociativa por conjuntos: ventajas y desventajas

- Combina lo mejor de las otras correspondencias
- La caché se divide en un grupo de conjuntos
    - Cada conjunto contiene un número de líneas
    - N vías con N = 2, 4, 8, etc.
- Un bloque determinado corresponderá a cualquier línea de un conjunto determinado

Política de reemplazos: Algoritmos de sustitución

Una vez que se ha llenado la caché, para introducir un nuevo bloque debe sustituirse uno de los bloques existentes.

★ En correspondencia directa:
- El que ocupa el lugar del nuevo
- No hay elección
- Sólo hay una posible línea para cada bloque
- Se necesita una sustitución de esa línea (si o si).

Para las correspondencias asociativas se requieren algoritmos de sustitución:

★ En correspondencia asociativa:
- **Los algoritmos deben implementarse en hardware** (para conseguir velocidad)
- LRU - ***Least Recently Used*** (Menos recientemente usado)
    - Es una política de reemplazo que expulsa la línea de caché que no ha sido usada desde hace más tiempo.
    - Se basa en el historial temporal de accesos: la línea menos “reciente” es la candidata a eliminar.
    - Requiere controles de tiempos
- FIFO - ***First In, First Out*** (Primero en entrar, primero en salir / el más antiguo)
    - Reemplaza la línea que ingresó primero al conjunto, independientemente de cuán frecuentemente o recientemente se haya usado.
    - Funciona como una cola: el más viejo se va.
    - Requiere controles de acceso
- LFU - ***Least Frequently Used*** (Menos frecuentemente usado)

        - Expulsa la línea con menor cantidad de accesos acumulados.
        - Se basa en un contador de frecuencia: la línea menos utilizada a lo largo del tiempo es la que se reemplaza.
        - Requiere controles de uso
        - Se sustituye aquella línea que ha experimentado menos referencias.
    - Reemplazo Aleatorio - *Random Replacement*
        - Selecciona una línea al azar dentro del conjunto para reemplazar, sin usar historia de accesos ni contadores.
        - Es muy simple y sorprendentemente efectivo en ciertas arquitecturas.

Política de escritura

Cuando se debe reemplazar un bloque de la caché, si se ha realizado al menos una operación de escritura sobre una palabra de la línea correspondiente, la memoria principal debe actualizarse reescribiendo la línea de caché en el bloque de memoria. **Se debe evitar inconsistencia de las memorias.**

Política de escritura: en acierto

Write-through *(Escritura inmediata)*.
- Se actualizan simultáneamente la posición de la caché y de la memoria principal.
    - se genera mucho tráfico y retrasa la escritura.

Write-back *(Post-escritura)*.
- La información sólo se actualiza en la caché.
    - Se marca como actualizada -> *bit de "sucio"*.
    - La memoria principal se actualiza en el reemplazo y puede contener información errónea en algún momento.

Política de escritura: en fallo

Write allocate
- La información se lleva de la memoria principal a la caché. Se sobreescribe en la caché
    - Habitual con write-back

No-write allocate
- El bloque no se lleva a la memoria caché. Se escribe directamente en la memoria principal.
    - Habitual con write-through

Clase 8: Procesadores superescalares

Un procesador superescalar es aquél que usa **múltiples cauces de instrucciones independientes**. Cada cauce consta de múltiples etapas, de modo que puede tratar **varias instrucciones a la vez**, por lo que saca provecho del **paralelismo** en las instrucciones. Capta varias instrucciones a la vez, y a continuación intenta encontrar instrucciones cercanas que sean independientes entre sí y puedan ejecutarse en paralelo.

Segmento de cauce

Una unidad segmentada es una secuencia de etapas con la propiedad de que nuevas operaciones pueden iniciarse mientras otras están en proceso

Mayores prestaciones: La evolución lógica de los diseños de cauces segmentados dio lugar a la aparición de dos técnicas de ejecución de mayores prestaciones.

- ➔ Procesadores Supersegmentados
- ➔ Procesadores Superescalares

Enfoque Supersegmentado

- Muchas operaciones no necesitan todo un ciclo de reloj.
- Obtendremos mayores prestaciones subdividiendo el ciclo de reloj en sub-intervalos
  - Resulta una mayor frecuencia del ciclo de reloj.
- División de las etapas “macro” del cauce segmentado en sub-etapas más pequeñas (y, por tanto, más rápidas) y se transmiten los datos a la mayor velocidad del ciclo de reloj.
- El tiempo para las instrucciones individuales no varía.
  - Aumenta el grado de paralelismo.
  - Incrementa la aceleración percibida.

En resumen: se dividen las etapas que ocupan más tiempo en sub-etapas, generando más instrucciones ejecutándose en simultáneo, pero sin mejorar el tiempo total de una instrucción.

Enfoque Superescalar

- Se pueden llevar a cabo (completar) más de una instrucción simultáneamente.
- Conlleva la duplicación de algunas o todas las partes de la CPU/ALU.
  - Captar múltiples instrucciones al mismo tiempo.
  - Ejecutar sumas y multiplicaciones simultáneamente.
  - Ejecutar carga/almacenamiento, mientras se lleva a cabo una operación en ALU.
- El grado de paralelismo y, por tanto, la aceleración de la máquina aumentan, ya que se ejecutan más instrucciones en paralelo.

En resumen: Se ejecutan más de una instrucción en simultáneo, a partir de la duplicación de ciertos componentes de la CPU/ALU. Así por ejemplo se pueden captar múltiples instrucciones o ejecutar operaciones numéricas simultáneas. Esto mejora el grado de paralelismo (más instrucciones ejecutándose en simultáneo).

Superescalar vs. supersegmentación

La **supersegmentación** aprovecha el hecho de que muchas etapas del cauce realizan tareas que requieren menos de medio ciclo de reloj. De este modo, se dobla la velocidad de reloj interna, lo que permite la realización de dos tareas en un ciclo de reloj externo. Por otro lado, un procesador **superescalar** puede ejecutar instrucciones en diferentes cauces de manera independiente y concurrente.

Estas mejoras son muy favorables, pero traen **nuevos conflictos en la ejecución**. Para que dos instrucciones se puedan ejecutar en paralelo sin ningún problema, deben ser independientes, es decir que la ejecución de una no dependa ni interfiera de ninguna forma con la otra.

Grado de paralelismo: Depende de la cantidad de instrucciones que se puedan ejecutar en simultáneo, y para tener un grado alto, se necesitan prestaciones de parte de la computadora (recursos duplicados, captación simultánea de instrucciones) y las instrucciones no deben generar dependencias entre sí, es decir ser independientes.

Paralelismo de instrucciones

Existe cuando instrucciones de una secuencia son independientes y entonces pueden ejecutarse en paralelo solapándose
El paralelismo a nivel de instrucciones es una técnica que permite que múltiples instrucciones se ejecuten en paralelo dentro de un procesador.

- Limitaciones:
  - **Dependencia de datos verdadera:** cuando una instrucción necesita leer datos que son el resultado de una instrucción previa
  - **Dependencias de Control:** Las dependencias de control se refieren a las instrucciones condicionales, como las instrucciones de salto condicional. El procesador no puede prever de antemano cuál será la próxima instrucción a ejecutar hasta que se resuelva la condición.
  - **Conflictos en los recursos:** es una pugna de dos o más instrucciones por el mismo recurso al mismo tiempo
  - **Limitaciones de Ancho de Banda de Memoria:** El acceso a memoria principal es una operación que puede restringir el paralelismo a nivel

de instrucción. Si el procesador necesita acceder repetidamente a la memoria, es posible que deba esperar a que se completen las operaciones de lectura/escritura antes de poder avanzar con otras instrucciones.

- **Dependencia de salida:** se produce cuando dos instrucciones tratan de escribir en un mismo registro
- **Antidependencia:** cuando una instrucción intenta escribir datos antes de que otra instrucción previa haya terminado de leer esos mismos datos.
- **Tamaño de Ventana de Ejecución:** La cantidad máxima de instrucciones que se pueden emitir y ejecutar en paralelo puede ser limitada por cuestiones de diseño de hardware o por la arquitectura del procesador

Paralelismo de la máquina

Es una medida de capacidad del procesador para sacar partido del paralelismo de instrucciones

- Depende de:
  - Número de instrucciones captadas por ciclo
  - Número de unidades funcionales
  - Mecanismos para localizar instrucciones independientes

Políticas de emisión en superescalares

- **Emisión y finalización en orden:** Se emiten las instrucciones en el orden exacto en que lo haría una ejecución secuencial, y se escriben los resultados en ese mismo orden.
- **Emisión en orden y finalización desordenada:** Las instrucciones se pueden completar en cualquier orden una vez que han sido ejecutadas, siempre y cuando no altere el resultado final. La emisión se para cuando hay cualquier tipo de conflicto o dependencia de datos.
- **Emisión y finalización desordenada:** Las instrucciones se emiten a la etapa de ejecución en el orden en que están listas para ser ejecutadas, y se pueden completar en cualquier orden, siempre y cuando no altere el resultado final.

Renombre de registros

El objetivo principal de utilizar la técnica de Renombre de Registros en un procesador superescalar es resolver conflictos de dependencia de almacenamiento de datos y permitir una ejecución más eficiente y paralela de instrucciones.

- Las dependencias de salida y antidependencias surgen porque los valores de los registros no pueden reflejar la secuencia de valores dictada por el flujo del programa.
    - Conflictos de almacenamiento
- Esto puede detener alguna etapa del cauce
- Cuando la ejecución de una instrucción guarda un resultado nuevo en Registro, se almacena en un registro nuevo
    - **Renombramiento de registros**
- Los registros se asignan dinámicamente
    - Las referencias posteriores son a los registros nuevos

Implementación superescalar

Características:

- ➢ Estrategias de captación simultánea de múltiples instrucciones
- ➢ Lógica para determinar dependencias verdaderas entre valores de registros y mecanismos para comunicar esos valores.
- ➢ Mecanismos para iniciar o emitir múltiples instrucciones en paralelo
- ➢ Recursos para la ejecución en paralelo de múltiples instrucciones.
- ➢ Mecanismos para entregar el estado del procesador en un orden correcto

---

Creo que de este tema preguntan hasta acá.

Consideraciones en el procesamiento superescalar

Excepciones en procesadores superescalares

- En un procesador superescalar varias instrucciones se ejecutan a la vez.
- Si una instrucción causa una excepción, puede pasar que algunas posteriores ya hayan terminado, esto lleva a un estado inconsistente y una excepción imprecisa.
- Pero el comportamiento debe ser como en un procesador normal, sin paralelismo.

Para que la excepción sea precisa:

1. Todas las instrucciones anteriores a la que falla deben completarse.
2. La que falla y todas las posteriores se abortan / descartan.
3. Luego de atender la excepción, el procesador reanuda desde la instrucción que falló.

Excepciones precisas

- Para lograr precisión, los resultados deben escribirse en el orden correcto (orden del programa).
- Ejemplo: retardar la escritura para que las instrucciones terminen en orden.
- Con interrupciones externas, también se exige precisión.

- El procesador hace:
    - La unidad de emisión deja de despachar instrucciones.
    - Se cancela la cola de instrucciones nuevas.
    - Las instrucciones pendientes terminan.
- Después de eso recién comienza la interrupción.

Compromiso clásico

Ejecución desordenada (para usar mejor el hardware) vs. **Finalización en orden** (para tener excepciones precisas).

Solución:
- Se ejecuta desordenado,
- Pero se finaliza en orden usando:
    - Etapa de escritura temporal
    - Renombramiento de registros
- Esto evita que una instrucción escriba un valor final hasta que sea seguro hacerlo (para mantener el orden y la precisión).

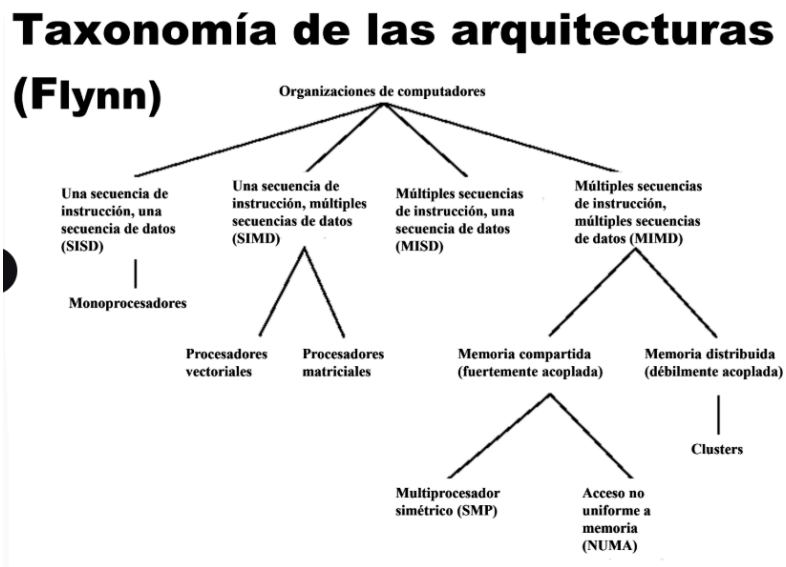
Clase 9: Procesamiento paralelo

Introducción al procesamiento paralelo:

Sea cual sea el nivel de prestaciones, la demanda de máquinas de mayor rendimiento seguirá existiendo.
- Mejorar el rendimiento de una máquina con un solo procesador
- Arquitecturas de sistemas con varios procesadores.

Taxonomía de Flynn

La Taxonomía de Flynn es una clasificación utilizada para categorizar las arquitecturas de computadoras según la cantidad de instrucciones y datos que pueden procesar simultáneamente.



Categorías de Computadoras

- SISD: Una sola unidad de procesamiento ejecuta una única secuencia de instrucciones que opera en un único conjunto de datos en cada ciclo de reloj.
- SIMD: Una sola instrucción se aplica simultáneamente a múltiples conjuntos de datos. Esto significa que múltiples unidades de procesamiento ejecutan la misma instrucción en diferentes conjuntos de datos al mismo tiempo.
- MISD: Múltiples secuencias de instrucciones se ejecutan en paralelo en un solo conjunto de datos
- MIMD: Cuentan con múltiples unidades de procesamiento, cada una ejecutando su propia secuencia de instrucciones en su propio conjunto de datos de manera independiente. Esto permite la ejecución de programas diferentes o partes diferentes de un programa en paralelo.

SISD

- Un único procesador interpreta una única secuencia de instrucciones (SI - *Secuencia de Instrucciones*).
- Datos almacenados en una única memoria (UM - *Memoria Única*)
- Computadoras monoprocesador

SIMD

- Una única instrucción máquina controla paso a paso la ejecución simultánea de un cierto número de elementos de proceso (EP - *Elemento de Proceso*)
- Cada elemento de proceso tiene una memoria dedicada (ML - *Memoria Local*)

- Cada instrucción es ejecutada por cada procesador, con un conjunto de datos diferentes.
- Procesadores vectoriales y matriciales

❖ Computadora con una única “unidad de control” y una matriz de elementos “computacionales”
❖ Tipos de instrucciones de procesador:
  ➢ Extensiones de las instrucciones escalares:
    ■ Sumar, almacenar, multiplicar, etc. Se convierten en operaciones vectoriales ejecutadas en todos los procesadores de modo simultáneo.
  ➢ Debe añadirse la capacidad de transferir al conjunto de instrucciones los datos escalares y vectoriales entre procesadores: atributos de un “lenguaje paralelo”.

MISD

- Se transmite una secuencia de datos a un conjunto de procesadores
- Cada procesador ejecuta una secuencia de instrucciones diferente
- Esta estructura nunca ha sido implementada

MIMD

- Un conjunto de procesadores ejecuta secuencias de instrucciones diferentes en simultáneo
- Conjuntos de datos diferentes
- Se pueden dividir según la forma de comunicarse
  - Memoria compartida
    - **SMP** (multiprocesadores simétricos) y sistemas NUMA
  - Memoria distribuida
    - **Clusters**

Multiprocesador simétrico - SMP: **Computadora autónoma (*Symmetric Multiprocessing*)**

- Dos o más procesadores similares de capacidades comparables
- Comparten la memoria principal y las E/S
- Interconectados mediante un bus u otro tipo de sistema de interconexión
- Tiempo de acceso a memoria similar para todos los procesadores (UMA - *Uniform Memory Access*)
- Todos los procesadores pueden desempeñar las mismas funciones
- Sistema operativo integrado, que proporciona la interacción entre los procesadores y sus programas

Ventajas potenciales de un SMP

- Mayores prestaciones
  - Si el trabajo a realizar puede originarse en paralelo
- Buena disponibilidad
  - Un fallo en un procesador no detendrá la computadora
- Crecimiento incremental
  - Se pueden añadir más procesadores
- Escalado
  - En función de la cantidad de procesadores
- **Cuidado: Bus compartido**

Bus de tiempo compartido: desventajas

- La prestación está limitada por el tiempo del ciclo del bus
- Cada procesador debería estar equipado con una memoria caché para mejorar las prestaciones
  - Se reducirá el número de accesos
- Se pueden producir problemas de coherencia de cache
  - Este problema debe ser resuelto por el hardware
    - Protocolos de sondeo y protocolos de directorio

Clusters

- Computadoras completas interconectadas que trabajan conjuntamente como un único recurso
  - Ilusión de que se trata de una única máquina
- Cada computadora se denomina “nodo”
- Prestaciones y disponibilidad elevadas
- Aplicaciones propias de un servidor
- Son la alternativa a los SMP

Beneficios del cluster

- Escalabilidad absoluta
- Escalabilidad incremental
- Alta disponibilidad
- Mejor relación precio / prestaciones

Cluster vs SMP

- Ambos:
  - Dan soporte a aplicaciones de alta demanda de recursos
  - Disponibles comercialmente (SMP es más antiguo)
- SMP:

    - Más fácil de administrar y configurar
    - Cercano a los sistemas de un solo procesador
        - La planificación (scheduling) es la diferencia principal
        - Menos espacio físico / menor consumo de potencia
- Cluster:
    - Superior escalabilidad incremental y absoluta
    - Superior disponibilidad
        - Redundancia

Hago una línea porque, si bien esto está en la teoría, a partir de acá no vi que pregunten estos temas en finales.

Términos UMA, NUMA, CC-NUMA

Todos los procesadores tienen acceso a toda la memoria

- UMA - *Uniform memory access*
    - Igual tiempo de acceso a todas las regiones de memoria
    - Igual tiempo de acceso a memoria para los diferentes procesadores
- NUMA - *Nonuniform memory access*
    - El tiempo de acceso de un procesador difiere dependiendo de la región de memoria que accede
    - Diferentes procesadores acceden a diferentes regiones de memoria a diferentes velocidades
- CC-NUMA - *caché coherente NUMA*
    - Es un NUMA que mantiene coherencia de cache entre las cache de los distintos procesadores

Motivación NUMA

- SMP tiene un límite práctico en su número de procesadores
- En clusters cada uno tiene su propia memoria principal
    - Aplicaciones “no ven” la memoria global
    - Coherencia de cache mantenida por software no por hardware
- NUMA retiene las características tipo SMP y brinda multi procesamiento a gran escala
- Objetivo NUMA: tener una memoria transparente del sistema y permitir nodos, cada uno con su propio bus o sistema de conexión interna

Operación CC-NUMA

- Cada procesador tiene caché L1 y L2
- Cada nodo tiene su propia memoria principal
- Nodos conectados por algún tipo de red
- Cada procesador ‘ve’ un único espacio de direcciones de memoria

- Orden de acceso a memoria:
  - cache L1 (local al procesador)
  - caché L2 (local al procesador)
  - Memoria principal (local al nodo)
  - Memoria remota
    - Petición por red
- Automático y transparente

Procesamiento Multihilos *(Multithreading)*

- Aumento de paralelismo de instrucciones
  - Sin el aumento de complejidad y consumo de potencia de la segmentación de cauce y los superescalares
- La secuencia de instrucciones se divide en secuencias más pequeñas llamadas hilos (threads) que pueden ejecutarse en paralelo
- Amplia variedad de diseños multihilo

Términos: hilo y proceso

- Concepto de hilo en procesadores multihilo puede no ser el de sistema operativo multiprogramado
- Proceso:
  - Un programa “corriendo” en una computadora
    - Propiedad de Recursos: espacio de direcciones virtuales para almacenar la imágen de un proceso (code, data, stack, etc)
    - Planificación / ejecución: hay camino de ejecución (traza)
- Conmutación de Proceso (*process switch*)
- Hilo:
  - Unidad de trabajo de un proceso que puede asignarse
  - Incluye un contexto de procesador (incluido PC y SP) y área de datos para su pila (stack)
  - Se ejecuta secuencialmente.
  - Interrumpible. El procesador cambiaría a otro hilo
- Conmutación de hilo (*thread switch*)
  - Cambio de control del procesador entre hilos de un mismo proceso
    - Usualmente menos costosa que la conmutación de proceso

Multihilo implícito y explícito

- Multihilo explícito
  - Ejecución concurrente de instrucciones de diferentes hilos explícitos
    - Mezcla de instrucciones de diferentes hilos en cauces compartidos
    - Ó por ejecución paralela en cauces paralelos

    - Todos los procesadores comerciales lo usan

- **Multihilo implícito**
    - Ejecución concurrente de varios hilos extraídos de un único programa secuencial.
        - Definidos estáticamente por el compilador ó dinámicamente por el hardware

Procesador multihilo

- ➢ PC (contador de programa) distinto para cada hilo que pueda ejecutarse concurrentemente.
    - Cantidad y tipo de HW para ejecución concurrente
- ➢ Se trata cada hilo separadamente
    - Predicción de saltos, renombre de registros y etc para optimizar la ejecución.
        - Paralelismo entre hilos
- ➢ Aproximaciones con ejecución simultánea real

Extra: BUSES 🚌

En las teorías de la materia no encontré nada sobre buses pero lo saqué de preguntas de finales viejos. Lo toman.

¿Qué es un bus?

Un bus es un camino de comunicación entre dos o más dispositivos. Se trata de un medio de transmisión compartido, se conectan varios dispositivos y cualquier señal transmitida por uno de ellos está disponible para que los otros puedan acceder a ella.
En una computadora se precisan buses de **tres tipos principales** (según la información que transportan):

1. **Datos:** Transmite todos los datos (instrucciones incluidas), el ancho del bus definirá el tamaño de la palabra de la arquitectura.
2. **Direcciones:** Identifica la ubicación de un dato en memoria, el ancho de este bus define la capacidad máxima de memoria en el sistema. Si el bus tiene n líneas puede direccionar 2^n posiciones de memoria.
3. **Control:** Transmite señales de control como timers, lectura/escritura o interrupciones

En un sistema con bus compartido (sólo un maestro a la vez), **solo un dispositivo puede transmitir datos en un momento dado**. Si dos dispositivos intentan usar el bus simultáneamente, se produciría una colisión.

Por eso se necesita un mecanismo de arbitraje, cuyo objetivo es:

- ➢ Determinar qué dispositivo será el maestro del bus, es decir, quién puede iniciar una transferencia de datos.
- ➢ El otro dispositivo actuará como esclavo.

Los dispositivos que pueden convertirse en maestro suelen ser:

- La CPU
- Un módulo E/S
- Un controlador DMA

Técnicas de arbitraje:

- ★ Arbitraje centralizado: Un único dispositivo hardware controla el acceso al bus, normalmente la CPU o el DMAC.
  - Los dispositivos que quieren usar el bus envían una solicitud.
  - Decide un árbitro

- ★ Arbitraje distribuido: No existe un controlador central, cada módulo tiene lógica propia para competir por el bus. Los módulos cooperan mediante protocolos para decidir quién lo usa.
  - Cada dispositivo anuncia su intención de usar el bus.
  - Los módulos se organizan entre sí.
  - El ganador se convierte en maestro y el otro en esclavo.

Tipos de buses: según cómo están físicamente organizadas las líneas del bus

- Dedicados: Un bus dedicado tiene líneas permanentemente asignadas a una función específica (datos, direcciones o control) o a un conjunto particular de dispositivos.
- Multiplexados: Un bus multiplexado utiliza las mismas líneas físicas para transmitir distintos tipos de información en diferentes instantes del tiempo. Por ejemplo, primero se envía la dirección y luego los datos, usando una señal de control que indica qué tipo de información está presente.
  - **Ventaja:** uso de menos líneas (ahorra espacio y costes).
  - **Desventaja:** necesita una circuitería más compleja en cada módulo.

Técnicas de temporización: La temporización o sincronización es la forma en la que se coordinan los eventos del bus.

- ➔ Sincrónica: La presencia de un evento en el bus está determinada por un reloj. El bus incluye una línea de reloj a través de la que se transmite una secuencia en la que se alternan intervalos regulares de igual duración a uno y a cero. Un único intervalo a uno seguido de otro a cero se conoce

como ciclo de reloj o ciclo de bus y define un intervalo de tiempo unidad (time slot).

- ➔ Asincrónica: No se utiliza el reloj, si no que los dispositivos se comunican entre sí para informar que se está realizando un evento. La presencia de un evento en el bus es consecuencia y depende de que se produzca un evento previo.

La temporización sincrónica es más fácil de implementar y comprobar, pero es menos flexible ya que todos los dispositivos deben usar la misma frecuencia de reloj.

Elementos característicos de un bus

- **Tipos de buses:** Las líneas del bus pueden ser dedicadas o multiplexadas
- **Método de arbitraje:** Puede ser centralizado o distribuido
- **Temporización:** Es la forma de coordinar los eventos en el bus, puede ser sincrónica o asincrónica.
- **Anchura del bus:** Está determinada por la cantidad de líneas que tiene el bus. En un bus de datos, la anchura determina cuántos bits se pueden transferir al mismo tiempo. En un bus de direcciones, la anchura del bus determina la máxima capacidad de memoria posible en el sistema
- **Tipo de transferencia de datos:** Todos los buses permiten tanto transferencias de escritura (dato de maestro a esclavo) como de lectura (dato de esclavo a maestro). En ciertos buses también son posibles algunas operaciones combinadas. Algunos buses permiten también transferencias de bloques de datos.

Bus PCI y SCSI

El bus PCI y el SCSI son dos medios de comunicación con los periféricos, el SCSI es utilizado principalmente para medios de almacenamiento como discos duros, mientras que el PCI se utiliza hoy en día para conectar distintos tipos de placas (de video por ejemplo) a la motherboard.

Bus SCSI: *Small Computer System Interface* (Pequeña interfaz del sistema de cómputo)

- ➢ Es una **interfaz estándar** para la transferencia de datos entre distintos dispositivos del bus de la computadora. Se utiliza para comunicar dispositivos rápidos, como discos CD-ROM, dispositivos de audio y dispositivos de almacenamiento externo de datos. **Requiere un controlador de interfaz.**

Bus PCI: *Peripheral Component Interconnect* (Interconexión de Componente Periférico)

- Es un bus de ancho de banda elevado, independiente del procesador, que se puede utilizar como bus de periféricos o bus para una arquitectura de entreplanta.
